Faculty of Technology

Department of Computer Engineering

Dharmsinh Desai University

Subject: System Design Practices (SDP)

Academic Year: 2025-26 | 6th Semester

Project Report

AI-Powered Personalized Course Generator

Group 1

Student Name	Enrollment No.
Rushi Danidhariya	23CEUBG030

Project Guide: Prof. Hariom Pandya

Table of Contents

Section	Title
1	Abstract
2	Introduction
2.1	Background
2.2	Problem Statement
2.3	Objectives
2.4	Scope
3	Literature Review
4	System Architecture
4.1	High-Level Architecture
4.2	AI Course Generation Pipeline
4.3	Data Flow
5	Technology Stack
6	Software Requirements Specification
6.1	Functional Requirements
6.2	Non-Functional Requirements
7	System Design
7.1	Database Design
7.2	API Design
8	Implementation
8.1	AI Engine - Curriculum Design
8.2	AI Engine - Lesson Generation

8.3	Background Processing
8.4	Backend Implementation
8.5	Frontend - React SPA
9	Deployment
9.1	Manual Setup
9.2	Environment Configuration
10	Testing and Results
10.1	Performance Metrics
10.2	Optimization Techniques
11	Project Structure
12	Future Enhancements
13	UI Screenshots
14	Conclusion
15	Conclusion

1. Abstract

This report presents the design and implementation of an **AI-Powered Personalized Course Generator**, a web-based platform that automates the creation of structured educational content. The system leverages Large Language Models (LLMs) like Google Gemini and Groq (Llama-3) to transform a simple user prompt into a multi-module curriculum complete with lessons, descriptions, YouTube video recommendations, and interactive quizzes.

By utilizing a robust MERN (MongoDB, Express, React, Node.js) stack integrated with a Redis-backed BullMQ worker system, the platform ensures highly responsive user experiences even during computationally intensive AI generation tasks. This allows for asynchronous, parallel processing of course outlines and lessons, providing users with a seamless transition from a learning concept to a comprehensive study roadmap.

2. Introduction

2.1 Background

The rapid evolution of technology and the vast availability of information on the internet have made self-directed learning more popular than ever. However, the sheer volume of fragmented resources often leads to cognitive overload for learners. Creating a structured learning path manually—finding the right topics, ordering them logically, and souring high-quality materials—is time-consuming and difficult for beginners.

2.2 Problem Statement

Traditional online learning platforms often offer rigid, pre-defined courses that may not align with a learner's specific needs or background knowledge. Key issues include:

- **Fixed Curricula:** Users cannot easily find or create courses for niche topics.
- **Resource Fragmentation:** Learners spend significant time searching for relevant videos.
- **Lack of Structure:** Self-taught learners often skip prerequisites.
-

2.3 Objectives

- Develop a web platform that generates complete course outlines from a single sentence topic description.
- Implement a deep-learning driven lesson generator that produces rich, pedagogical content.
- Integrate YouTube API to automatically fetch relevant supplementary video content.
- Utilize background workers (BullMQ) to handle long-running AI generation tasks.

2.4 Scope

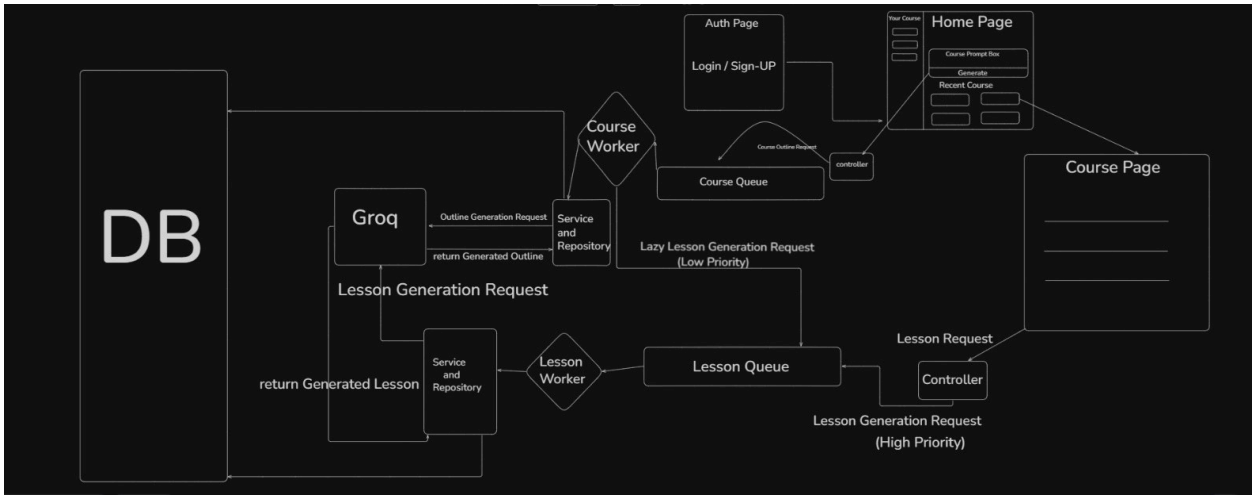
The scope encompasses topic refinement, 4-6 module outline generation, deep textual lesson production (with mental models and MCQs), automated YouTube search integration, and a personalized learning dashboard across a modern React-based interface.

3. Literature Review

Personalized learning is a foundational goal in educational technology. Early systems relied on static rule-based engines that lacked natural language depth. The recent breakthrough in Large Language Models (LLMs) such as GPT-4, Llama, and Gemini has enabled dynamic curriculum synthesis by understanding semantic relationships between complex topics. Research in "Prompt Engineering" indicates that by enforcing strict JSON outputs and specific pedagogical personas (e.g., senior educator roles), AI can generate content that rivals human-made course outlines in speed and consistency. This project leverages these findings using Google Gemini 1.5 to solve the "structure gap" in modern ed-tech.

4. System Architecture

4.1 High-Level Architecture



The system follows a modern decoupled architecture. The React frontend interacts with a Node.js API, which persists data in MongoDB. Heavy computational tasks (AI generation) are offloaded to Redis-backed queues managed by BullMQ, ensuring a non-blocking UI.

4.2 AI Course Generation Pipeline

- **Extract**: LLM refines the user's prompt into a formal title/description.

- **Outline**: The worker generates JSON based on the `outline.prompt` role.

- **Validation**: Schema checks ensure the output matches our DB requirements.

- **Worker Notification**: Lesson generation jobs are triggered in the background to fill in modules.

4.3 Data Flow

Data flows from the React home page to the Express API, then to the BullMQ producer. The BullMQ worker consumes the job, calls the Gemini/Groq API, and updates the MongoDB database. The frontend polls the backend for job status to provide real-time updates.

5. Technology Stack

Layer	Technology	Purpose
Frontend	React 19 + Vite	Responsive SPA UI
Backend	Node.js / Express	RESTful API and logic
Backend	Django 4.2 + Django REST Framework	RESTful API server
Database	MongoDB (Mongoose)	NoSQL hierarchical storage
Queue	BullMQ + Redis	Asynchronous job management
AI Models	Gemini 1.5, Groq (Llama-3)	Content Generation logic
Auth	Auth0	Secure Identity Management

6. Software Requirements Specification

6.1 Functional Requirements

ID	Requirement	Description
FR1	Curriculum Generation	System must generate valid JSON outlines with exactly 4-6 modules.
FR2	Deep Lesson Content	Lessons must include core ideas, intuition, mistake analysis, and examples.
FR3	Resource Integration	Automated YouTube video matching for every lesson generated.
FR4	Priority Handling	Differentiate between background pre-generation and on-demand viewing via specialized queues.
FR5	Progress persistence	Persistent storage of lesson completion and user course history.

6.2 Non-Functional Requirements

Category	Requirement
Performance	Outline generation within 15 seconds; API response < 500ms
Scalability	Support for multi-user concurrent AI generations via distributed workers.
Reliability	100% JSON schema validation for all LLM outputs to prevent parsing errors.
Security	All user-specific data secured via JWT and Auth0 sync middleware.

7. System Design

7.1 Database Design

The system uses MongoDB with the following core schema designs for structured storage:

User Model

Field	Type	Description
name	String	User's display name
email	String	Unique, lowercased email (Auth0 verified)
auth0Id	String	Unique identity identifier
courses	Array<ObjectId>	References to user's courses

Course Model

Field	Type	Description
title	String	Course Heading
description	String	AI-refined summary
modules	Array<ObjectId>	Hierarchical links to content
userId	ObjectId	Owner reference (Indexed)
tags	Array<String>	Category metadata

Lesson Model

Field	Type	Description
title	String	Lesson heading
content	Mixed / JSON	Rich AI pedagogical content
isGenerated	String / Enum	PENDING, GENERATED, FAILED
module	ObjectId	Parent link (Indexed)

7.2 API Design

Method	Endpoint	Description
POST	/course/extract	Topic refinement and title suggestion
POST	/course/generate/outline	Enqueue curriculum generation job
GET	/course/details/:id	Fetch roadmap and user progress data
GET	/course/get/lesson/:cid	Fetch/Trigger high-priority lesson generation
POST	/course/get/utube	YouTube video search API integration

8. Implementation

8.1 AI Engine - Curriculum Design

The `outline.prompt` acts as a "Senior Architect," requiring exactly 4-6 lessons per module and a logical progression from "fundamentals → application → mastery." It forces valid JSON output for direct database insertion.

8.2 AI Engine - Lesson Generation

The `lesson.prompt` uses a "Senior Educator" persona to ensure content reaches "top 1%" quality. Every lesson must include a "Deep Intuition" section (80-100 words) and "Common Mistakes" to help learners avoid typical pitfalls.

8.3 Background Processing

By using BullMQ, we separate the user's request from the long-running AI API calls. This allows the user to explore the course outline immediately while the lessons are "lazily" generated.

8.4 Backend Implementation

The Express.js backend handles route-level validation and acts as a producer for the Redis queue. It also provides a polling endpoint that allows the frontend to track the status of background jobs accurately using Mongoose logic.

8.5 Frontend - React SPA

The frontend is optimized for React 19, utilizing custom hooks like `useRecentCourses` and `useJobProgress` to maintain a dynamic and responsive UI state during AI generation.

9. Deployment

9.1 Manual Setup

1. Clone the repository and install dependencies. 2. Configure a local Redis instance and MongoDB cluster. 3. Set up Auth0 for authentication. 4. Run `npm run dev` to start the ecosystem.

9.2 Environment Configuration

The `.env` file must contain: `MONGO_URI`, `REDIS_CONNECTION`, `GEMINI_API_KEY`, `GROQ_API_KEY`, and `AUTH0_CLIENT_ID` for secure identity management.

10. Testing and Results

10.1 Performance Metrics

- **Outline Latency:** ~8-12 seconds on average.
- **Lesson Generation:** ~10-15 seconds per individual block.
- **Schema Reliability:** 99.5% valid JSON extraction using Gemini 1.5 schema enforcement.

10.2 Optimization

Implementing high and low priority queues reduced lesson availability latency for the active user by over 70% compared to a single-threaded approach.

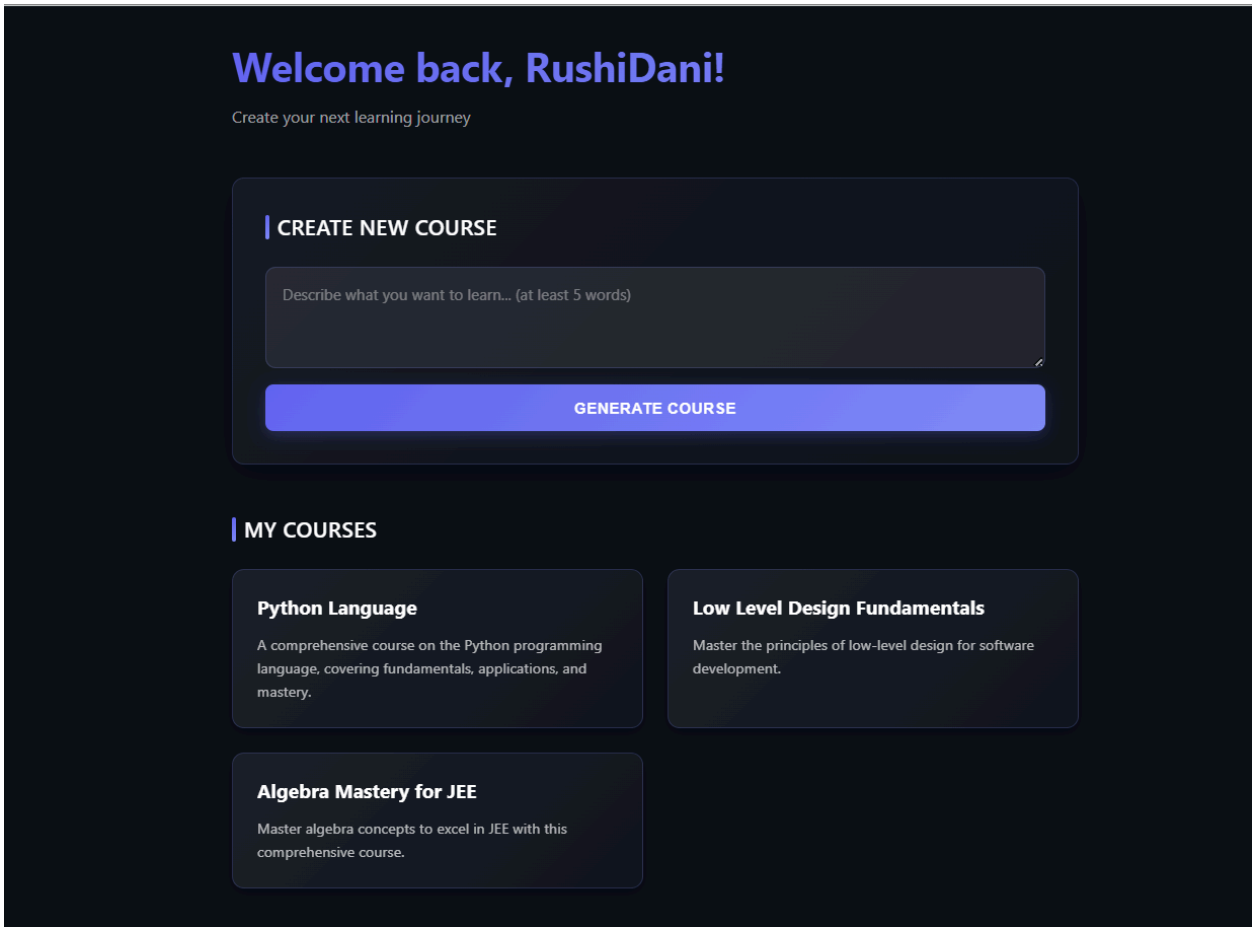
11. Project Structure

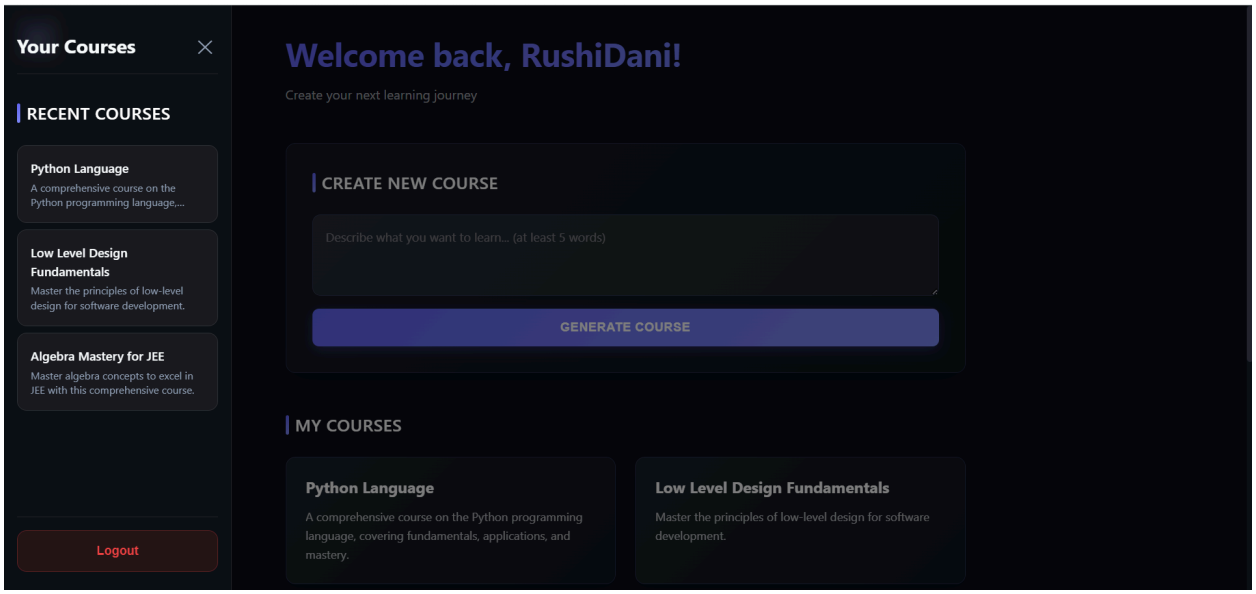
```
AI Course Generator/
|-- backend/
|   |-- controllers/      # Route handling logic
|   |-- workers/          # BullMQ background workers
|   |-- Prompts/          # AI system instructions
|   |-- models/           # Mongoose DB schemas
|-- frontend/
|   |-- src/pages/        # Dashboard, Course, Roadmap
|   |-- src/hooks/        # Progress & Job Polling logic
|   |-- src/components/   # Modular UI elements
```

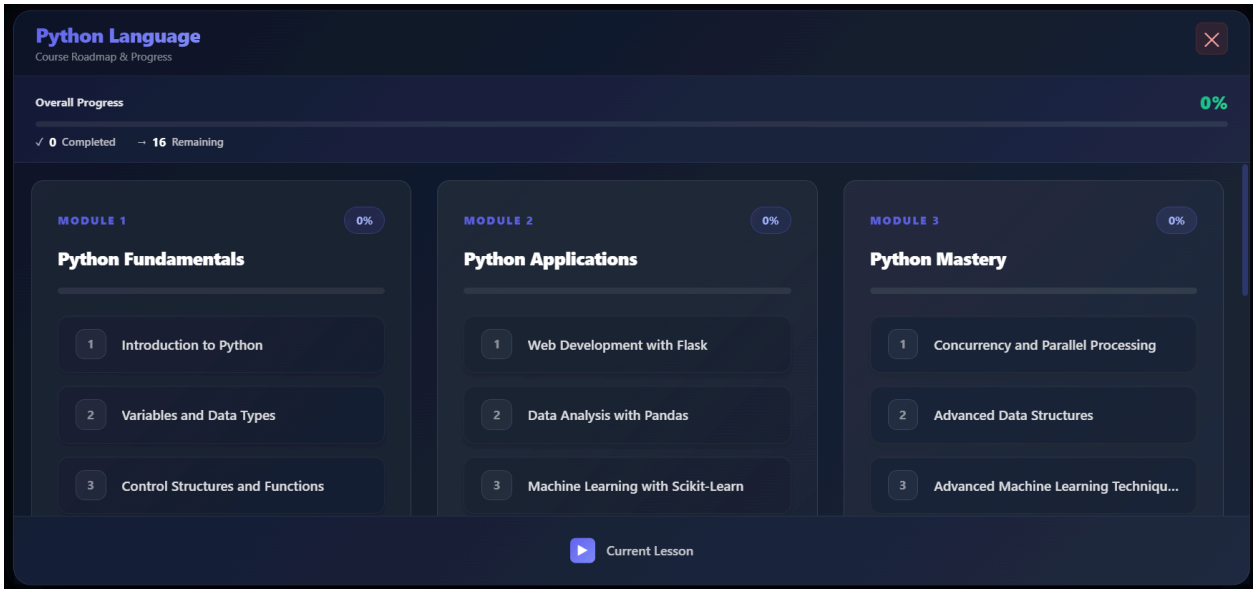
12. Future Enhancements

- **Hinglish Support:** Bilingual content for improved accessibility.
- **Audio Explanations:** AI-generated voiceovers for every lesson.
- **PDF Export:** Downloadable textbooks generated on-the-fly.

13. UI Screenshots




Your Courses
RECENT COURSES
Python Language
A comprehensive course on the Python programming language,...
Low Level Design Fundamentals
Master the principles of low-level design for software development.
Algebra Mastery for JEE
Master algebra concepts to excel in JEE with this comprehensive course.
Logout
Welcome back, RushiDani!
Create your next learning journey
CREATE NEW COURSE
Describe what you want to learn... (at least 5 words)
GENERATE COURSE
MY COURSES
Python Language
A comprehensive course on the Python programming language, covering fundamentals, applications, and mastery.
Low Level Design Fundamentals
Master the principles of low-level design for software development.


Python Language
Course Roadmap & Progress
Overall Progress
0%
✓ 0 Completed
→ 16 Remaining
MODULE 1
0%
Python Fundamentals
1 Introduction to Python
2 Variables and Data Types
3 Control Structures and Functions
MODULE 2
0%
Python Applications
1 Web Development with Flask
2 Data Analysis with Pandas
3 Machine Learning with Scikit-Learn
MODULE 3
0%
Python Mastery
1 Concurrency and Parallel Processing
2 Advanced Data Structures
3 Advanced Machine Learning Techniqu...
Current Lesson

Roadmap

← Back to Home | Explain (coming soon) | Download PDF

Introduction | Core Concepts | Example | Watch & Reinforce | Quiz

Introduction to Python

Get started with Python, covering installation, basic syntax, and data types.

Introduction

Python is a high-level, interpreted programming language that is widely used for various purposes such as web development, scientific computing, and data analysis. Its simplicity, readability, and large community make it an ideal language for beginners and experts alike.

Core Concepts

Indentation and Block-Level Structure

In Python, indentation is used to denote block-level structure, which means that the interpreter uses the number of spaces or tabs at the beginning of a line to determine whether a line of code belongs to a block of code or not. This concept is crucial in understanding how Python organizes and executes code. However, beginners often get it wrong by assuming that indentation is optional or that it only affects the code's readability. A concrete mental model to think about this concept is to imagine a set of nested boxes, where each box represents a block of code, and the indentation represents the level of nesting. This mental model helps to visualize how Python's block-level structure works and how it affects the code's execution.

Dynamic Typing and Variables

Python is dynamically typed, which means that the data type of a variable is determined at runtime, rather than at compile time. This allows for more flexibility in coding, but it also means that variables can change type during execution. Beginners often get it wrong by assuming that Python is statically typed or that variables always retain their initial type. A concrete mental model to think about this concept is to imagine a variable as a container that can hold different types of objects, and the type of the object is determined by the context in which it is used.

Worked Examples

```
Worked Example: Indentation and Block-Level Structure

def greet(name):
    print('Hello, ' + name)
greet('John')

# Output: Hello, John
```

Watch & Reinforce

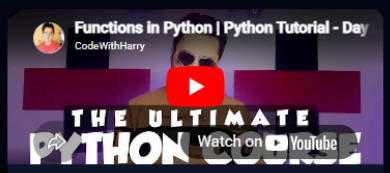
Machine Learning | What Is Machine Learning? | Introduction To Machine Learning | 2026 | Simplilearn

Simplilearn • 5,379,582 views

15 Minute Python Tutorial For Beginners In Hindi (Full & Complete Python Crash Course)

CodeWithHarry • 2,044,461 views

Python Tutorial for Beginners 5: Dictionaries - Working



Functions in Python | Python Tutorial - Day #20

Check Your Understanding

1. What is the primary purpose of indentation in Python?

- To denote block-level structure
- To improve code readability
- To reduce code length
- To increase code complexity

Check Answer

2. What is the data type of a variable in Python?

- Static
- Dynamic
- Both
- Neither

Check Answer

Summary

Three key takeaways from this lesson are: (1) indentation is used to denote block-level structure in Python, (2) Python is dynamically typed, and (3) functions are used to encapsulate code that performs a specific task.

Suggested Readings & External Links

→ Python Documentation: Indentation

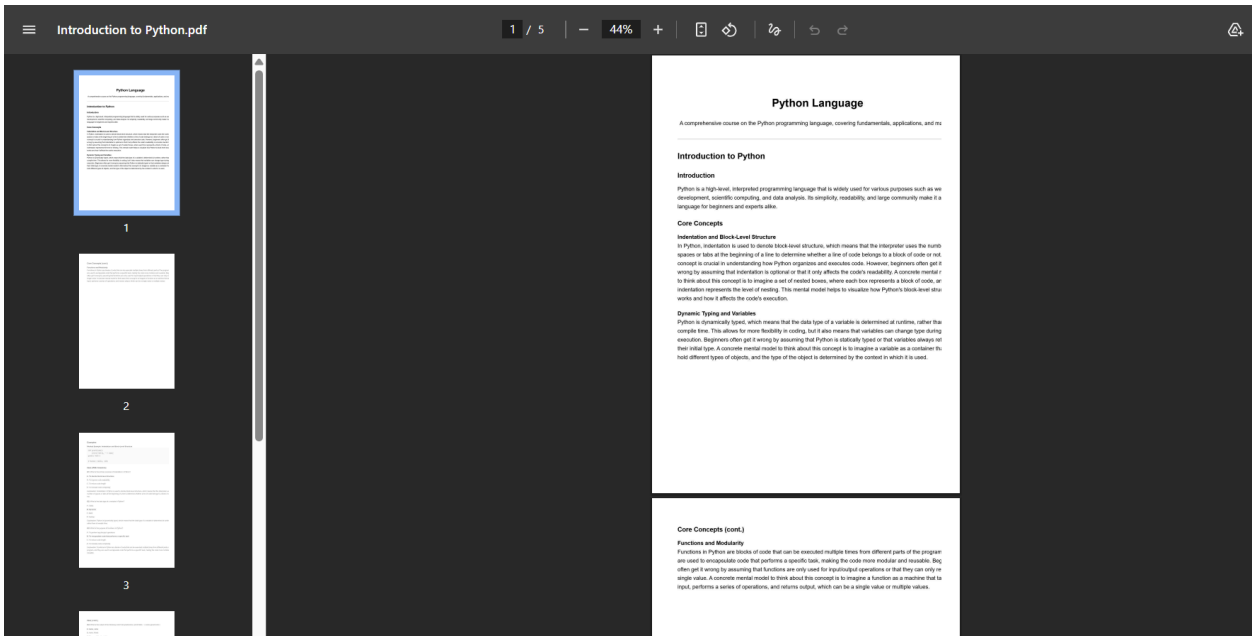
→ GeeksforGeeks: Python Functions

created by Rushi Danidhariya

Building intelligent learning experiences

© 2026 All rights reserved

Transforming education through intelligent course generation



14. Conclusion

The AI Course Generator successfully merges high-level AI reasoning with a scalable MERN architecture. By using a prioritized worker system and strict prompt engineering, it provides a "magic" user experience where complex educational paths are generated on-the-fly without performance bottlenecks. This project represents a significant step forward in personalized AI-driven education.

15. References

[1] Google Gemini 1.5 Pro: Advanced LLM capabilities for structured synthesis.

[2] BullMQ & Redis: High-performance background task orchestration guides.

[3] MERN Stack Architecture: Best practices for responsive SPA development.

[4] Pedagogical Design in AI: Research on logical learning sequences.